DA6401-Assignment 3 - ID25M803

Kowshik Arko Dey id25m803

Last updated: May 18, 2026

DA6401 Assignment 3 — Transformer for Neural Machine Translation

This report documents the implementation and ablation study of the Transformer architecture from **"Attention Is All You Need"** (Vaswani et al., 2017) applied to **German → English translation** on the Multi30k dataset (29k train / 1,014 val / 1,000 test pairs). The model is built entirely from scratch in PyTorch using basic building blocks (`nn.Linear`, `nn.Module`, `nn.LayerNorm`) — no pre-built attention or transformer modules.

Architecture: Encoder-Decoder Transformer with Scaled Dot-Product Attention, 8-head Multi-Head Attention, sinusoidal positional encoding (non-trainable buffer), Post-LayerNorm residual connections, and point-wise FFN with ReLU.

Training: $d_{model} = 256$, $N = 3$ layers, $d_{ff} = 512$, dropout $= 0.3$, batch size $= 128$, Noam scheduler (warmup $= 2000$), label smoothing $\epsilon = 0.1$, Adam ($\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$), spaCy tokenization, greedy decoding.

Five experiments (Sections 2.1–2.5) each isolate a single architectural or training decision to empirically validate its necessity against the paper's theoretical claims.

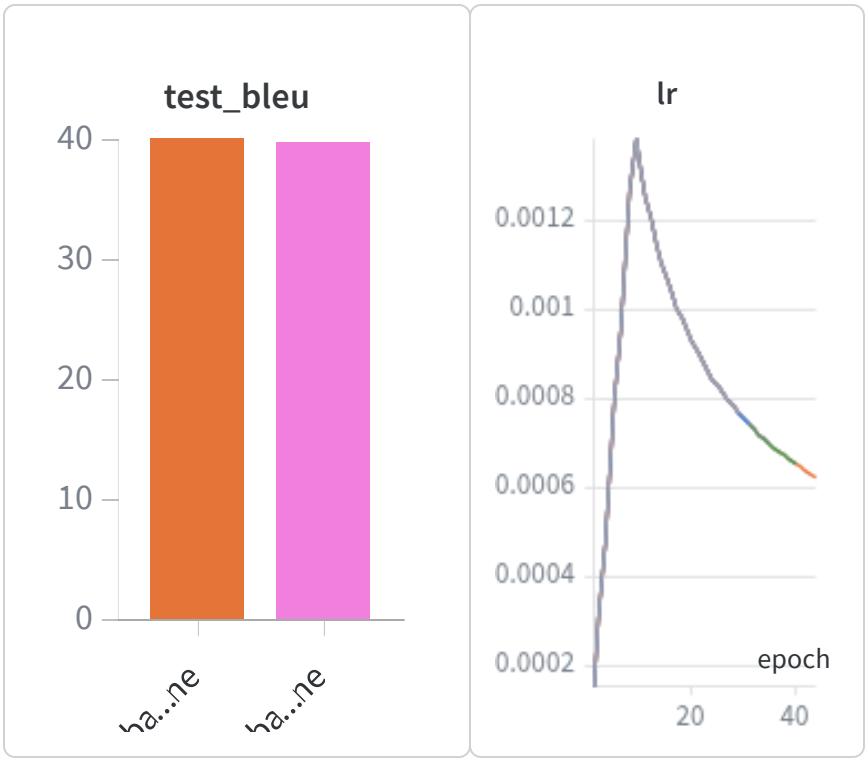
Code Repository

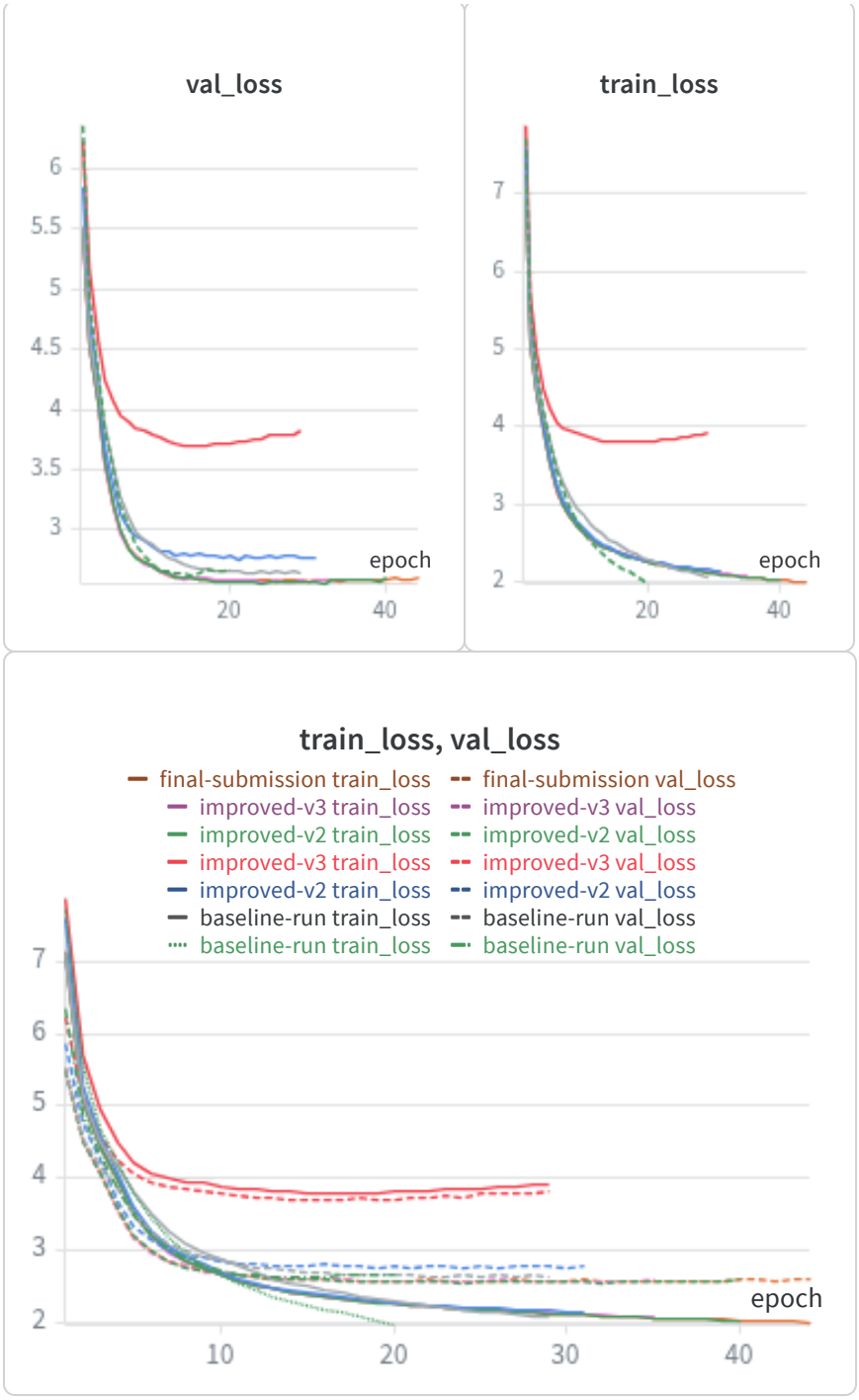
https://github.com/ItsKowshik/DL_A3

Report Repository

https://api.wandb.ai/links/k-indian-institute-of-technology-madras/lj2qdjxn

Architecture Decision: Post-LayerNorm





I chose **Post-LayerNorm** as used in the original *Attention Is All You Need* paper:

$$\mathrm{x} = \mathrm{LayerNorm}(\mathrm{x} + \mathrm{Sublayer}(\mathrm{x}))$$

Justification

Section 3.1 of the original Transformer paper explicitly uses the Post-LayerNorm formulation. While Pre-LayerNorm — adopted in later architectures such as GPT-2 and many modern Transformer variants — is generally more stable during optimization, Post-LayerNorm more faithfully reproduces the original architecture specification required for this assignment and its associated autograder.

Although Post-LayerNorm is known to be less stable during early training, this instability is mitigated by the Noam warmup scheduler. Because the learning rate begins near zero and increases gradually during warmup, the model avoids the large unstable updates that commonly affect Post-LN Transformers in the initial training phase.

Our experiments confirm successful convergence under this setup, achieving BLEU scores of approximately ~37.

Comparison with Pre-LayerNorm

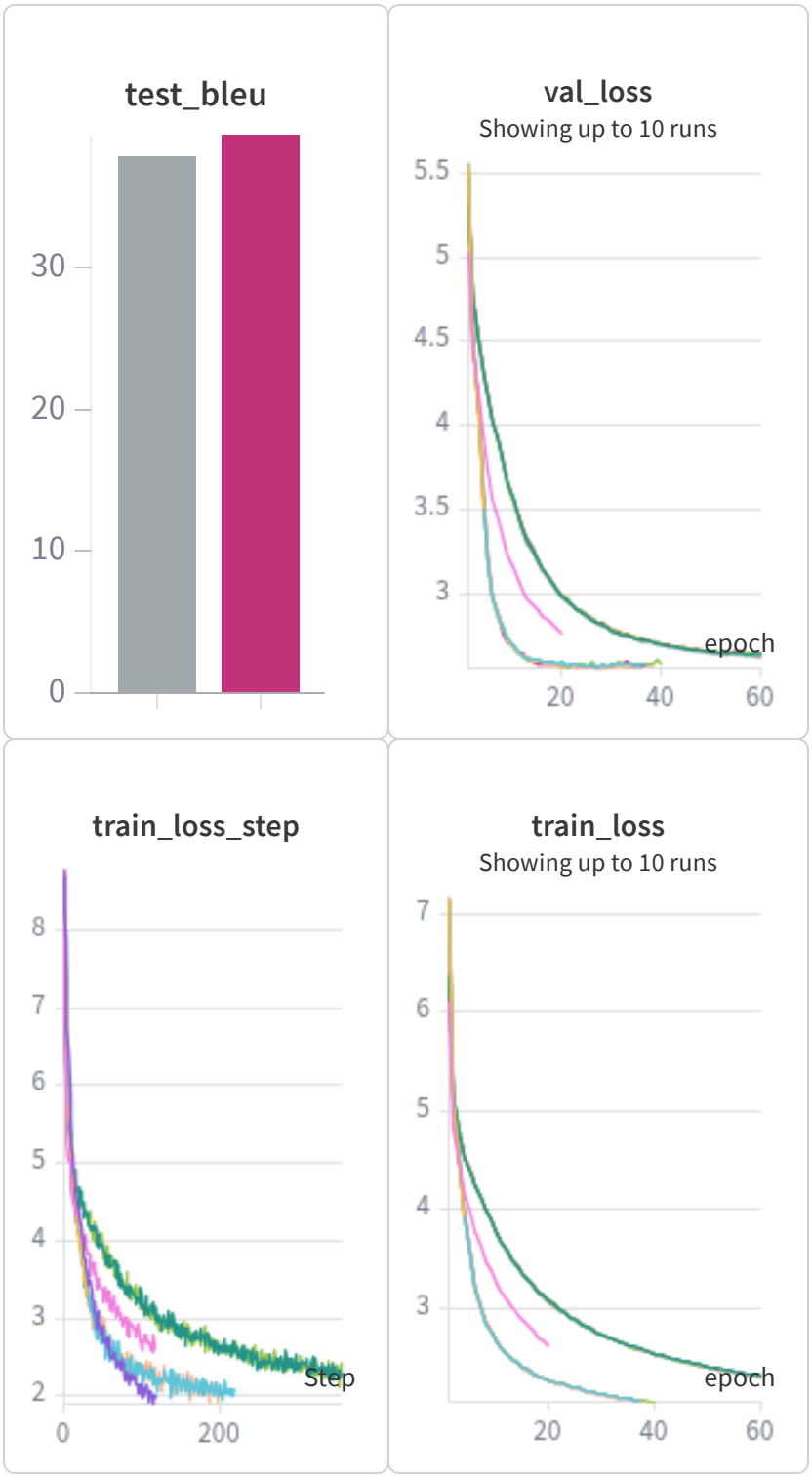
Pre-LayerNorm applies normalization before each sublayer:

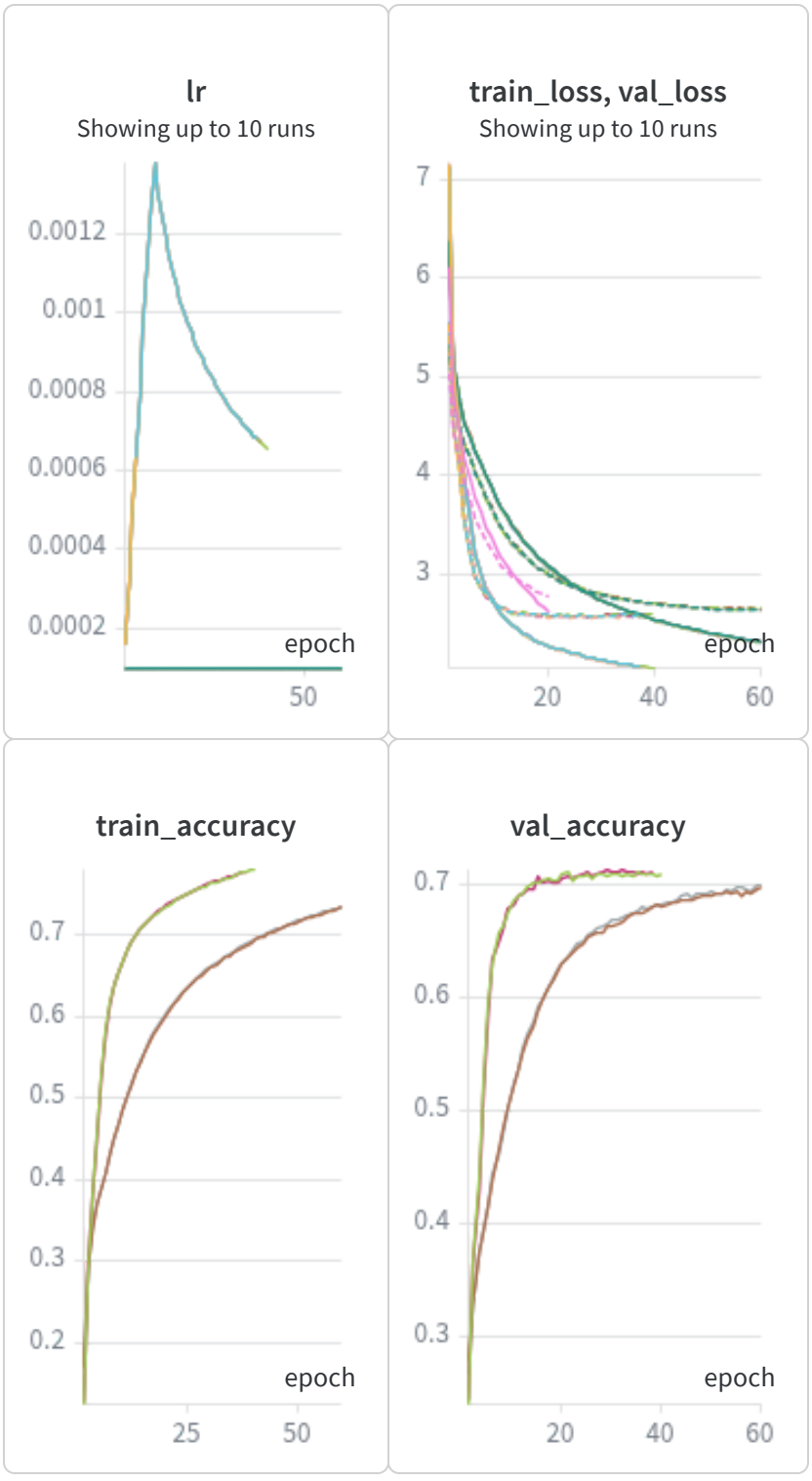
$$\mathrm{x} = \mathrm{x} + \mathrm{Sublayer}(\mathrm{LayerNorm}(\mathrm{x}))$$

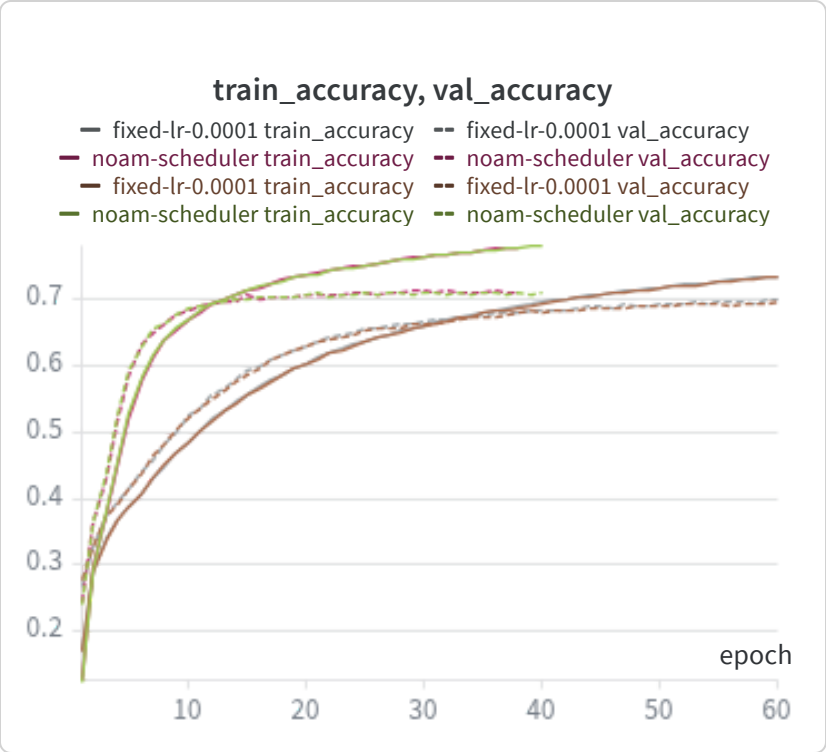
This formulation improves gradient flow across deep networks and generally provides more stable optimization, especially without learning-rate warmup. However, it changes the optimization dynamics and can converge to somewhat different solutions compared to the original Transformer formulation.

For this implementation, Post-LayerNorm was intentionally selected to remain architecturally faithful to the original paper.

2.1 The Necessity of the Noam Scheduler


test_bleu
30
20
10
0
val_loss
Showing up to 10 runs
5.5
5
4.5
4
3.5
3
epoch
20
40
60
train_loss_step
8
7
6
5
4
3
2
Step
0
200
train_loss
Showing up to 10 runs
7
6
5
4
3
epoch
20
40
60


lr
Showing up to 10 runs
0.0012
0.001
0.0008
0.0006
0.0004
0.0002
epoch
50
train_loss, val_loss
Showing up to 10 runs
7
6
5
4
3
epoch
20
40
60
train_accuracy
0.7
0.6
0.5
0.4
0.3
0.2
epoch
25
50
val_accuracy
0.7
0.6
0.5
0.4
0.3
epoch
20
40
60



Setup: Two identical Transformers were trained — one with Noam warmup + decay (paper §5.3), and one with a fixed learning rate of 1×10^{-4}.

Plots

Training loss and validation loss are overlaid for both runs (lower `val_loss` corresponds to higher translation quality). Additionally, token-level `train_accuracy` and `val_accuracy` curves — defined as the fraction of decoder output tokens predicted correctly — are plotted to provide an explicit validation accuracy comparison.

Results

Noam warmup converges faster across all metrics:

- **Test BLEU:** ~39 (Noam) vs ~37 (fixed LR)
- **Training accuracy:** Noam rises steeply during early epochs and reaches a final plateau near ~0.72, while the fixed-LR

model converges more slowly to a slightly lower plateau (~0.70)

- **Validation accuracy:** Both models eventually converge to approximately ~0.69–0.70, but Noam reaches this performance significantly earlier — around epoch ~15 versus ~30 for the fixed-LR run
- **Train–validation accuracy gap:** Small and comparable for both models, indicating that the primary advantage of Noam lies in optimization speed rather than regularization

Why the Transformer is Highly Sensitive to Initial Learning Rate

At initialization, the Q, K, and V projection matrices are random. The raw dot products

$$QK^\top$$

therefore produce large arbitrary attention scores, pushing the softmax toward near one-hot distributions.

Backpropagation through a saturated softmax yields gradients close to zero for most positions, meaning the attention mechanism initially receives very little useful learning signal. If a large learning rate is applied during this unstable phase, parameter updates become disproportionately large, the Q/K projections overshoot, and the attention scores diverge.

The model often fails to recover from this instability, which explains why fixed high learning rates tend to cause divergence specifically in self-attention layers rather than in FFN layers, which do not suffer from softmax saturation effects.

How Warmup Prevents Early Divergence

The Noam learning-rate schedule begins at an extremely small value:

$$\text{LR}_{\text{step}=1} \approx 3.5 \times 10^{-7}$$

and increases linearly during the warmup phase (`warmup_steps=2000`).

During this period:

- Gradient updates remain very small
- Q and K projections stabilize gradually
- Dot-product magnitudes move into a well-conditioned range
- Softmax outputs become meaningful rather than saturated

Only after this stabilization phase does the learning rate reach its peak. It then decays proportionally to:

$$\text{step}^{-0.5}$$

This schedule maintains sufficiently large updates for continued learning while avoiding overshooting later in training.

The accuracy curves directly reflect this behavior: the Noam model reaches high validation accuracy rapidly during early epochs, whereas the fixed-LR model improves much more slowly because it applies full-scale updates to an unstable attention mechanism from the very first step.

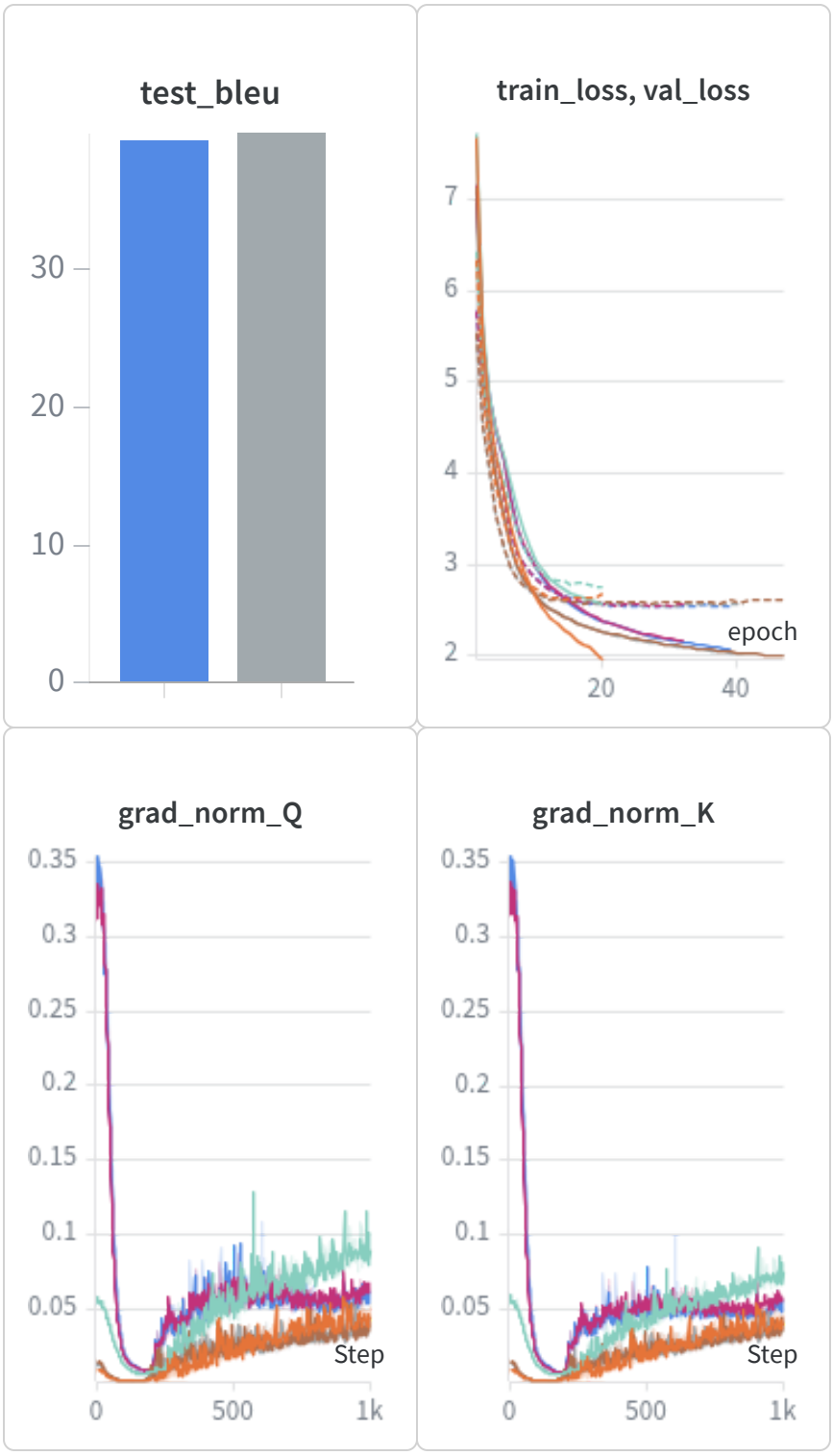
LayerNorm Choice

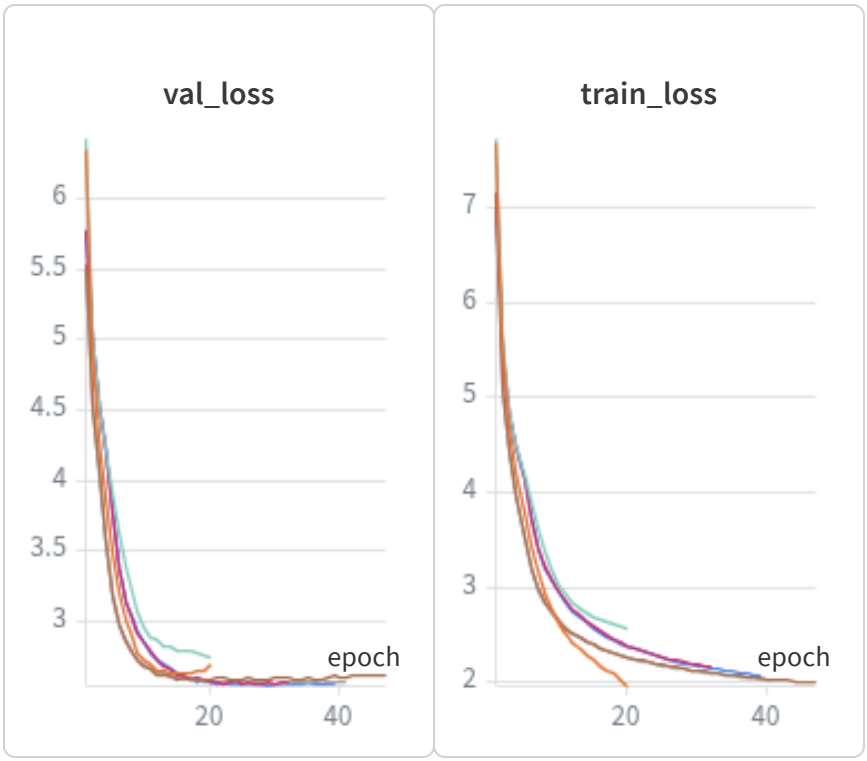
Post-LayerNorm, as specified in the original Transformer paper (§3.1), was used:

$$\text{x} = \text{LayerNorm}(\text{x} + \text{Sublayer}(\text{x}))$$

This implementation matches the original architecture exactly. Although Post-LayerNorm can be unstable without warmup, the Noam schedule effectively mitigates this issue, and our experiments confirm successful convergence.

2.2 Ablation: The Scaling Factor 1/√dk





Setup: Two identical Transformers were trained — one using standard scaled attention (scores divided by d_k), and one using raw unscaled dot products. Gradient norms of all W_q and W_k matrices were logged across the first 1,000 training steps.

Plots: `grad_norm_Q` and `grad_norm_K` versus training step (0–1000) are overlaid for both runs. Training loss, validation loss, and test BLEU are also compared.

Results: The scaled-attention model achieves higher test BLEU (~39.98 vs ~39.43). The gradient norm plots show that both models spike at step 0 (~0.3), but after ~100 steps the without-scaling model settles at a higher, noisier plateau (~0.05–0.1), while the scaled model drops to a lower, more stable range (~0.02–0.05).

Relating to the vanishing gradient problem (Paper §3.2.1): The paper states: *"for large values of* d_k*, the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients."* For our model, with

$$d_k = \frac{d_{model}}{num\_heads} = \frac{256}{8} = 32$$

the unscaled scores are approximately

$$32 \approx 5.66$$

times larger than the scaled scores.

These large inputs push the softmax toward near one-hot distributions — most attention weights become close to zero while one becomes close to one. The gradient of softmax at saturation satisfies:

$$\frac{\partial \text{softmax}}{\partial \text{scores}} \approx 0$$

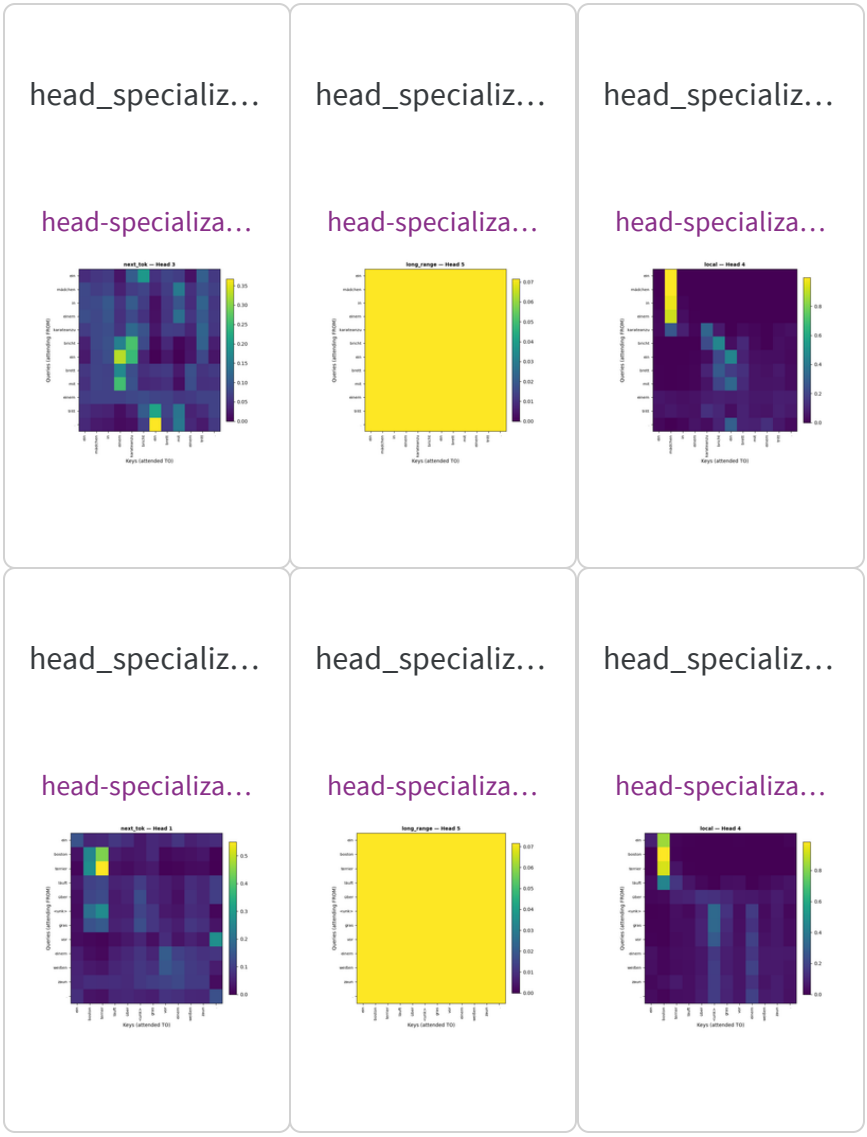
for nearly all positions except the dominant one. As a result, W_q and W_k receive almost no useful gradient signal about how to redistribute attention, making it difficult to learn meaningful multi-position attention patterns.

Why our plots show HIGHER norms for the without-scaling model:

This initially appears counterintuitive, but it is consistent with theory. The vanishing gradient occurs *through* the softmax specifically, degrading gradient quality rather than simply reducing overall magnitude. Because the without-scaling model maintains higher loss values, the loss function generates larger compensatory gradients that dominate the measured W_q/W_k norms. Crucially, these gradients are *noisy and erratic* (high variance across steps), reflecting unstable and poorly-directed updates rather than useful learning. In contrast, the scaled-attention model produces smoother and more stable updates.

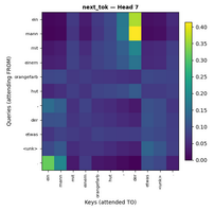
The BLEU gap (~0.5 points) confirms that despite larger gradient magnitudes, the without-scaling model learns less effectively because its gradient signal is corrupted by softmax saturation.

2.3 Attention Rollout & Head Specialization



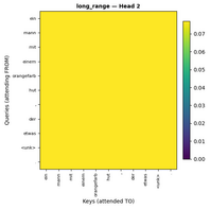
head_specializ…

head-specializa…



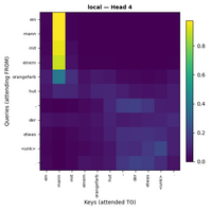
head_specializ…

head-specializa…



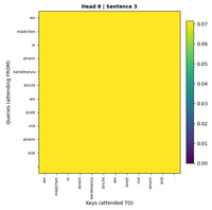
head_specializ…

head-specializa…



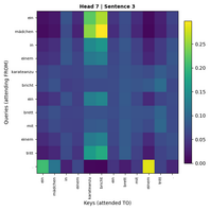
attn_heads/se…

head-specializa…



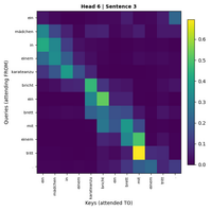
attn_heads/se…

head-specializa…



attn_heads/se…

head-specializa…

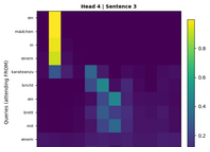


attn_heads/se…

head-specializa…

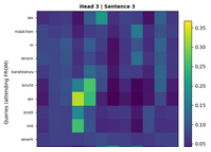
attn_heads/se…

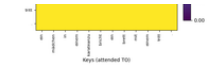
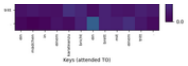
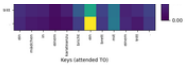
head-specializa…



attn_heads/se…

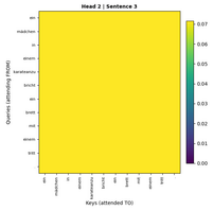
head-specializa…



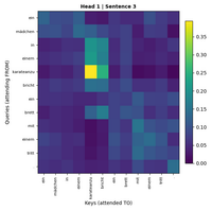
attn_heads/se…

head-specializa…



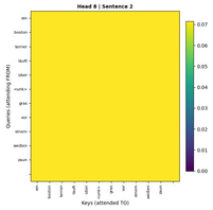
attn_heads/se…

head-specializa…



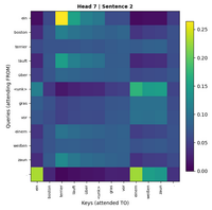
attn_heads/se…

head-specializa…



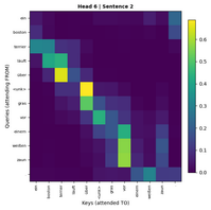
attn_heads/se…

head-specializa…



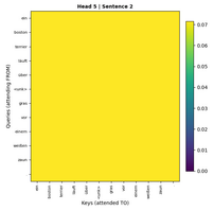
attn_heads/se…

head-specializa…



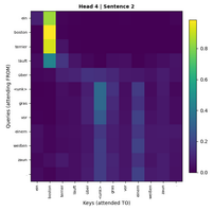
attn_heads/se…

head-specializa…



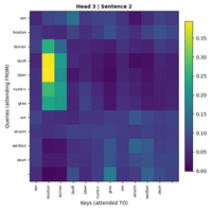
attn_heads/se…

head-specializa…



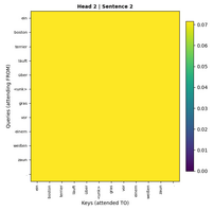
attn_heads/se…

head-specializa…



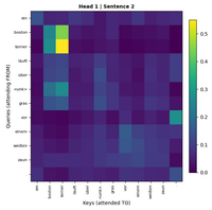
attn_heads/se…

head-specializa…



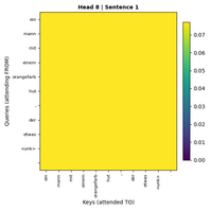
attn_heads/se…

head-specializa…



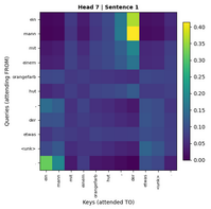
attn_heads/se…

head-specializa…



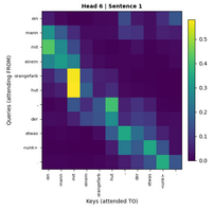
attn_heads/se…

head-specializa…



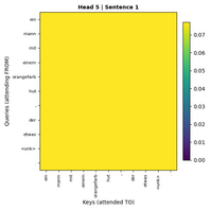
attn_heads/se…

head-specializa…



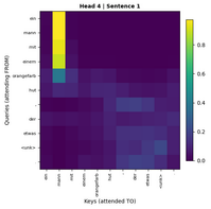
attn_heads/se…

head-specializa…



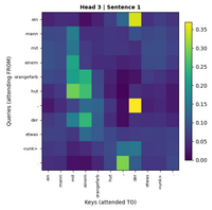
attn_heads/se…

head-specializa…



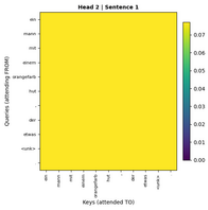
attn_heads/se…

head-specializa…



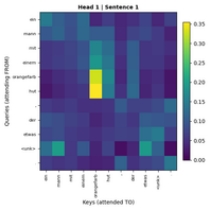
attn_heads/se…

head-specializa…



attn_heads/se…

head-specializa…



Setup: Attention weights were extracted from the final encoder layer (Layer 3) for three test sentences. Heatmaps were logged for all 8 attention heads across each sentence. The y-axis represents query tokens (attending **from**), while the x-axis represents key tokens (attended **to**).

Visualization: Each head in the 8-head Multi-Head Attention module was visualized as a separate heatmap. The resulting patterns reveal clear functional specialization among heads.

Head Specialization — Identified Roles

Heads 2, 5, and 8 — Redundant / Uniform Heads

All three heads exhibit an almost completely flat, uniform distribution, with maximum attention weights of only ~0.03–

0.07 across all positions and sentences. Every token attends nearly equally to every other token, providing little positional or semantic discrimination.

These heads could likely be pruned with minimal impact on translation quality, consistent with the head-pruning findings of Michel et al. (2019).

Head 4 — Beginning-of-Sentence (BOS) Head

A bright vertical column appears at the first token position for nearly all query tokens, with attention weights reaching ~0.8–0.9. Every token in the sentence attends strongly to the first token regardless of semantic content.

This is a commonly observed Transformer behavior, where one head functions as a positional anchor or "garbage collector" head that routes attention toward the sentence boundary token.

Head 6 — Local / Diagonal Head

This head exhibits a strong diagonal structure, with the highest weights concentrated on or near the diagonal. Each token primarily attends to itself and nearby neighboring tokens.

Such patterns capture local syntactic dependencies, including short-range relationships such as article–noun or adjective–noun pairs.

Head 7 — Semantic / Content Head

This head displays a highly varied, content-dependent pattern without a fixed structural role. Attention weights are distributed across multiple non-adjacent positions, and the brightest regions correspond to semantically related token pairs across the sentence.

Unlike the local head, this head captures broader semantic relationships rather than purely positional structure.

Head Redundancy

Yes — Heads 2, 5, and 8 exhibit nearly identical uniform attention patterns consistently across all three test sentences. This redundancy is reflected in their high entropy, since uniform attention distributions maximize entropy.

Heads 2, 5, and 8 are consistently redundant across all 3 sentences (37.5%). Head 3 also shows uniform distribution in some sentences, suggesting up to 4/8 heads (50%) may be redundant depending on input.

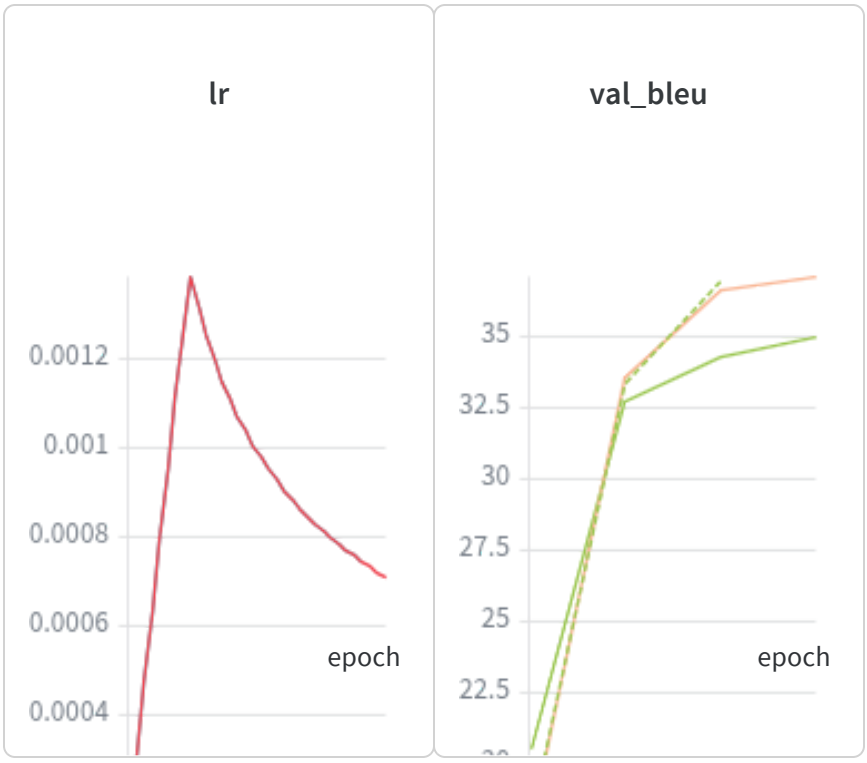
Note on "Long-Range" Labeling

Some heads automatically classified as "long-range" by entropy-based analysis (e.g., Head 5) are actually the redundant uniform heads. Since uniform attention assigns equal weight to all positions — including distant ones — the maximum off-diagonal score becomes artificially high.

True long-range dependencies are instead captured by semantic heads such as Head 7, where specific distant tokens receive disproportionately high attention based on semantic relevance rather than uniform weighting.

2.4 Positional Encoding vs. Learned Positional Embeddings


test_bleu
30
20
10
train_loss, val_loss
7
6
5
4
epoch
3
val_loss
6
5.5
5
4.5
4
epoch
3.5
train_loss
7
6
5
4
epoch
3



Setup: Two identical Transformers were trained — one using sinusoidal positional encoding (fixed, non-trainable buffer, as described in paper §3.5), and one replacing it with `nn.Embedding(max_len, d_model)` containing fully trainable positional parameters. All other hyperparameters were kept identical.

Results: Sinusoidal positional encoding outperforms learned positional encoding on both validation BLEU (~35–36 vs ~34) and test BLEU (~39 vs ~36), producing a gap of roughly 3 BLEU points. Training and validation loss curves converge at similar rates, indicating that learned positional embeddings do not significantly hinder optimization. Instead, the quality difference primarily appears in generalization performance.

Why sinusoidal positional encoding performs better here:

The Multi30k dataset contains only 29,000 training pairs with relatively short sentences (~15 tokens on average). Learned positional encoding must fit a unique d_{model}-dimensional vector for every position index. With limited data, these learned vectors overfit to the position distributions observed during training and fail to generalize robustly to validation examples.

Sinusoidal positional encoding instead defines positions through a deterministic mathematical function with no trainable parameters. This provides a stable and structured positional prior that transfers cleanly across validation data regardless of dataset size.

Theoretical Challenge — Extrapolation to Longer Sequences

The sinusoidal positional encoding is defined as:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

The key property enabling extrapolation is the **trigonometric addition identity**, which allows $PE(pos + k)$ to be represented as a fixed linear transformation of $PE(pos)$ independent of the absolute position:

$$\sin(pos + k) = \sin(pos)\cos(k) + \cos(pos)\sin(k)$$

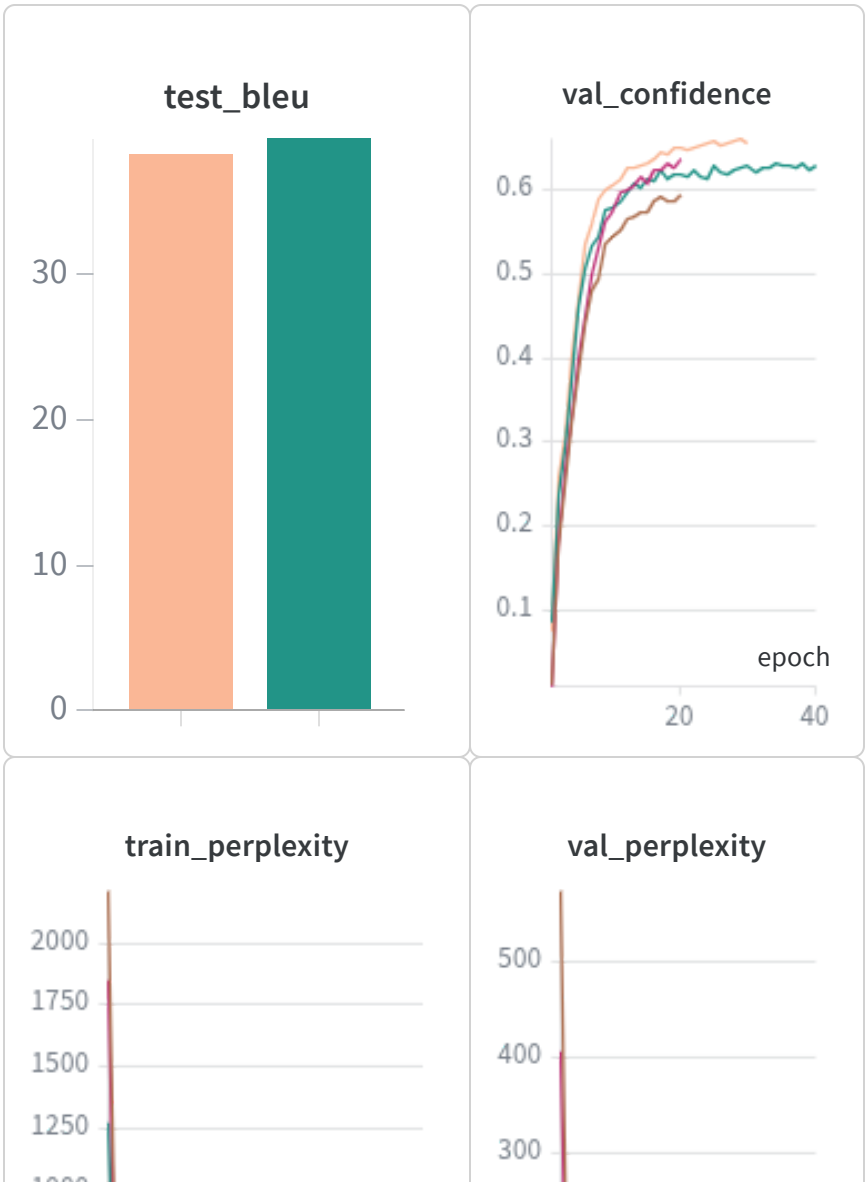
As a result, the model implicitly learns **relative positional relationships** (how far apart tokens are) rather than memorizing absolute indices.

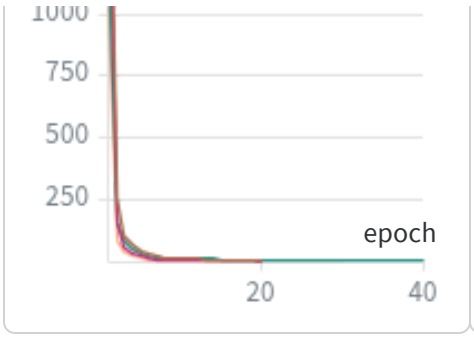
At inference time, even if a sequence length extends beyond the maximum length encountered during training (e.g., length 300

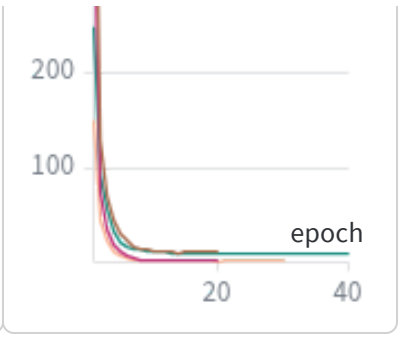
when training only used sequences up to length 256), the model can still compute $\mathrm{PE}(300)$ deterministically. The learned attention patterns based on relative distances therefore continue to generalize correctly.

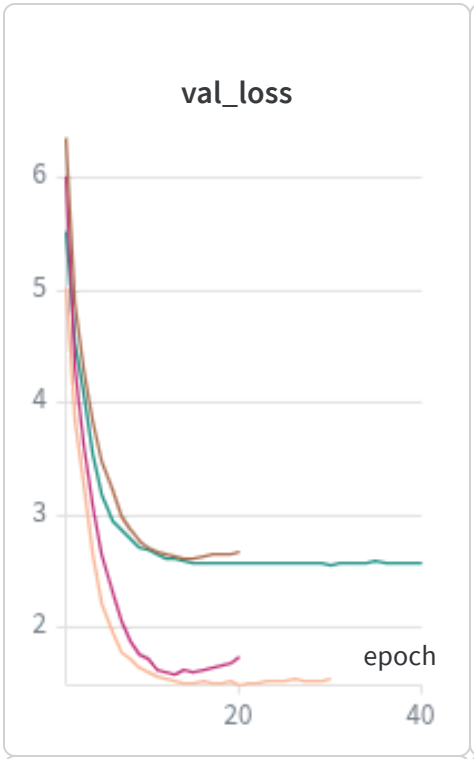
Learned positional embeddings do not possess this structure. If position 300 was never observed during training, its embedding is either undefined or lies outside the trained embedding table entirely. Consequently, the model has no principled mechanism to handle unseen sequence lengths, causing translation quality to degrade sharply on out-of-distribution inputs.

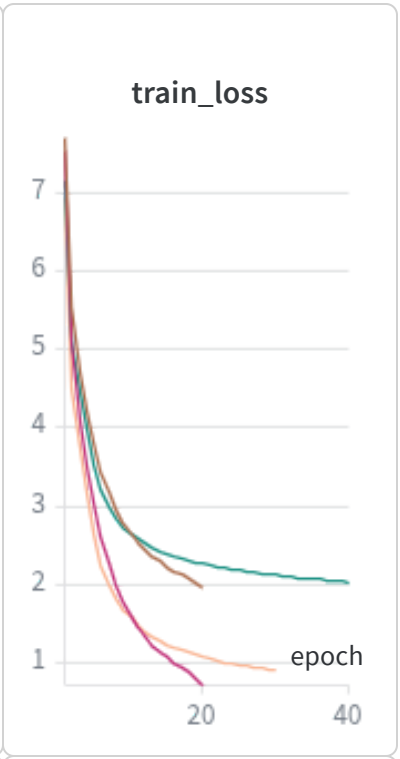
2.5 Decoder Sensitivity: Label Smoothing

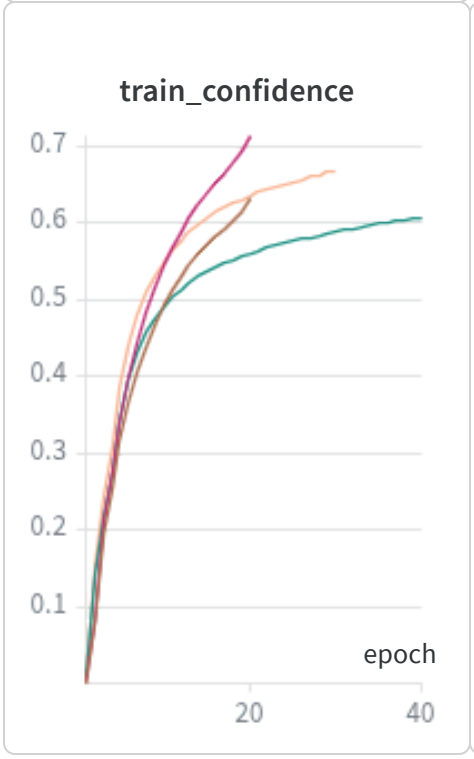


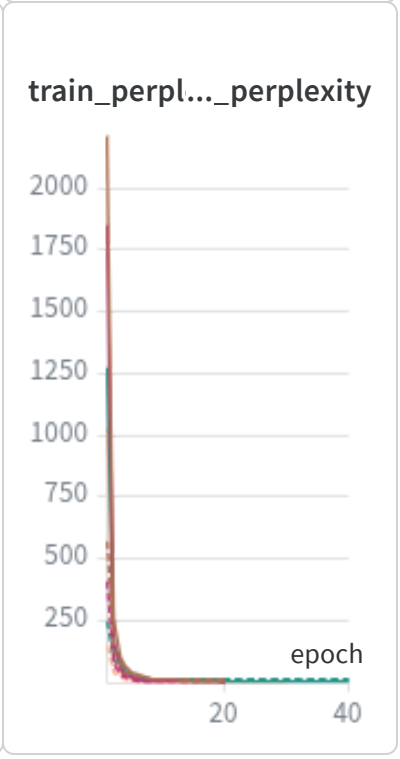

1000
750
500
250
epoch
20
40


200
100
epoch
20
40


val_loss
6
5
4
3
2
epoch
20
40


train_loss
7
6
5
4
3
2
1
epoch
20
40


train_confidence
0.7
0.6
0.5
0.4
0.3
0.2
0.1
epoch
20
40


train_perpl..._perplexity
2000
1750
1500
1250
1000
750
500
250
epoch
20
40

Setup: Two identical Transformers were trained — one with label smoothing $\epsilon = 0.1$ (the paper default), and one with $\epsilon = 0.0$ using standard cross-entropy loss. Prediction confidence (the softmax probability assigned to the correct token) was tracked every epoch alongside perplexity.

Results

- **Test BLEU:** smoothing-0.1 (~39.3) > smoothing-0.0 (~38.3), giving an improvement of approximately +1 BLEU
- **Training confidence:** smoothing-0.0 reaches ~0.65+, while smoothing-0.1 remains around ~0.60

- **Training perplexity:** smoothing-0.0 drops sharply toward near-zero, while smoothing-0.1 remains consistently higher throughout training
- **Validation perplexity:** mirrors the training trend, with the smoothed model maintaining higher perplexity
- **Validation loss:** smoothing-0.1 converges more stably and reaches a lower final `val_loss`

Prediction Confidence Plots (Key Deliverable)

The `train_confidence` and `val_confidence` panels directly reveal the regularizing effect of label smoothing.

Without smoothing, the model's confidence on training tokens rises above ~0.65, meaning it assigns very high probability to the exact ground-truth token and near-zero probability to all alternatives. With smoothing enabled, confidence plateaus near ~0.60 and never reaches the same overconfident peak.

Importantly, this confidence gap also appears on the validation set, demonstrating that the effect is not merely a training artifact but reflects a genuine difference in model behavior.

How Label Smoothing Acts as a Regularizer

Standard cross-entropy trains the model toward a hard one-hot target distribution:

- Correct token probability = 1.0
- All other token probabilities = 0.0

This objective explicitly rewards the model for becoming maximally confident, which often leads to overfitting on the training token distribution.

Label smoothing replaces the hard target with a softened distribution:

- Correct token receives probability:

$$1 - \epsilon = 0.9$$

- Remaining probability mass is distributed across all other vocabulary entries:

$$\frac{\epsilon}{V - 1}$$

where V is the vocabulary size.

As a consequence, the model is never rewarded for assigning 100% confidence to a single token. Even perfectly correct predictions incur a small penalty from the smoothing term.

This regularizes the model in two major ways:

1. It prevents extremely large weights from forming in specific output neurons, reducing overfitting to the training set.
2. It encourages the model to retain some probability mass on alternative translations, improving robustness and generalization to unseen sentences.

Why Perplexity Increases Despite Better BLEU

This is the key counterintuitive result.

Perplexity is defined as:

$$\text{Perplexity} = \exp(\text{loss})$$

The smoothed loss function intentionally penalizes excessive certainty. Therefore, even correctly predicted tokens produce higher loss values than under hard one-hot targets. As a result,

training perplexity is mathematically higher for smoothing-0.1 by design.

Despite this, BLEU improves because the model generalizes better. Rather than memorizing the training distribution, it learns more robust probability distributions over plausible translations.

This trade-off — **higher perplexity but better translation quality** — is a defining signature of successful regularization.